EECS 3404 Major Project Idea

Proposed Title

Explainable and Drift-Aware Machine Learning for Network Intrusion Detection

1. Project Overview

The goal of this project would be to build a machine-learning system that can classify network traffic as either:

- Normal traffic
- Malicious or attack traffic

We would use a public cybersecurity dataset containing examples of legitimate network activity and cyberattacks.

The project would compare several machine-learning models and study which model provides the best balance between detecting attacks and avoiding unnecessary false alarms.

This is only a proposed idea. The final topic, dataset, models, and scope can be decided together as a group.

2. Main Research Question

Which machine-learning model provides the best balance between detecting malicious network traffic and minimizing false-positive alerts?

3. Proposed Scope

To keep the project manageable, we could begin with binary classification:

- Class 0: Normal traffic
- Class 1: Attack traffic

Possible models to compare:

- Logistic Regression
- Random Forest
- XGBoost or Gradient Boosting
- Neural Network

4. Proposed Machine-Learning Workflow

Data preparation

We would:

- inspect and clean the dataset
- handle missing and duplicate values
- encode categorical features
- scale numerical features where necessary
- examine the class distribution
- identify possible data-leakage risks

All preprocessing would be learned only from the training data.

Validation strategy

The dataset would be divided into:

- training set
- validation set
- test set

The training set would be used to fit the models, the validation set would be used for tuning and model selection, and the test set would be kept unseen until the final evaluation.

Cross-validation could also be used for more reliable comparisons.

Class-imbalance handling

Since malicious traffic may be less common than normal traffic, we could compare techniques such as:

- class weighting
- oversampling or SMOTE
- threshold adjustment

Any resampling would be applied only to the training data.

5. Model Evaluation

The models could be evaluated using:

- Accuracy
- Precision
- Recall
- F1-score

- ROC-AUC
- PR-AUC
- Confusion matrix

Accuracy would not be used alone because a model could obtain high accuracy by predicting most traffic as normal while missing attacks.

6. Advanced Analysis

To give the project greater technical depth, we could include the following components.

Threshold tuning

Instead of automatically using a probability threshold of 0.50, we would compare different thresholds to find a suitable balance between:

- catching more attacks
- reducing false alarms
- avoiding missed attacks

False-positive and false-negative analysis

We would study:

- normal traffic incorrectly classified as an attack
- attack traffic incorrectly classified as normal
- patterns found in the misclassified examples
- which examples are most difficult for the models

Explainability

We could use SHAP or feature-importance methods to explain:

- which features influence predictions the most
- why a particular connection was classified as an attack
- why certain predictions were incorrect

Model calibration

We could examine whether the predicted probabilities are reliable.

For example, when a model predicts an 80% probability of an attack, we would check whether approximately 80% of similar cases are actually attacks.

Ablation analysis

We could conduct controlled experiments such as:

- with and without class-imbalance handling
- default threshold versus tuned threshold
- all features versus selected features
- calibrated versus uncalibrated predictions

This would help identify which parts of the system actually improve performance.

Drift testing

We could test what happens when the data changes, such as:

- training on one portion and testing on another portion
- changing the proportion of attack traffic
- modifying selected feature distributions
- testing on a later or different subset

This would help us evaluate whether the model could remain reliable after deployment.

7. Reproducibility

The project could include:

- organized Python code
- fixed random seeds
- reusable preprocessing and training functions
- a requirements file
- README instructions
- saved results and visualizations
- a repeatable training and evaluation process

8. Limitations and Failure Modes

Possible limitations to discuss include:

- the public dataset may not fully represent real company traffic
- attack examples may be old or simulated
- unseen attack types may not be detected
- too many false alerts could overwhelm security analysts
- model performance may decrease when network behaviour changes
- encrypted traffic may hide useful information

9. Expected Deliverables

The final project could include:

- cleaned dataset and preprocessing pipeline
- multiple trained ML models
- model-comparison tables
- confusion matrices
- ROC and Precision–Recall curves
- threshold and error analysis
- SHAP or feature-importance explanations
- calibration analysis
- ablation experiment
- drift experiment
- final report
- presentation video with a code and concept walkthrough

10. Connection to the Course Expectations

The proposed project would allow us to demonstrate:

- technical understanding
- proper ML methodology
- meaningful preprocessing
- leakage prevention
- validation strategy
- model comparison
- experimental rigor
- diagnostics and interpretation
- explainability
- calibration and uncertainty analysis
- ablation analysis
- engineering reasoning
- reproducibility
- critical evaluation
- limitations and failure-mode analysis

Group Discussion

This proposal is intended only as a starting point. The group can discuss and modify:

- the project topic
- the dataset
- the models
- the advanced experiments
- the final scope
- the division of responsibilities

Any other project ideas are also welcome before the group makes a final decision.